

Conceptos Básicos de Patrones de Diseño Simples en Java

Informe Detallado:

1. Introducción a los Patrones de Diseño

Los patrones de diseño son **soluciones comprobadas y recurrentes a problemas habituales en el diseño de software**. No son una porción específica de código que se pueda copiar directamente, sino más bien **"un concepto general para resolver un problema particular"**. A diferencia de los algoritmos, que definen un grupo claro de acciones para lograr un objetivo, un patrón es una descripción de más alto nivel de una solución, donde "el orden exacto de la implementación depende de ti."

Aprender sobre patrones es beneficioso porque:

- Proporcionan un "juego de herramientas de soluciones comprobadas" a problemas comunes en el diseño de software.
- Enseñan a resolver problemas utilizando principios del diseño orientado a objetos.
- Establecen un **"lenguaje común"** para una comunicación más eficiente entre desarrolladores, permitiendo frases como "Oh, utiliza un Singleton para eso", donde todos entienden la idea sin necesidad de explicaciones extensas.

Una descripción formal de un patrón generalmente incluye:

- **Propósito:** Breve explicación del problema y la solución.
- **Motivación:** Explicación detallada del problema y la solución que brinda el patrón.
- **Estructura de las clases:** Muestra las partes del patrón y sus relaciones.
- **Ejemplo de código:** Facilita la asimilación del patrón en un lenguaje de programación popular.

2. Clasificación de los Patrones

Los patrones se clasifican según su complejidad, nivel de detalle y escala de aplicabilidad:

- **Idioms:** Patrones más básicos y de bajo nivel, normalmente aplicados a un único lenguaje de programación.
- **Patrones de Diseño:** Soluciones probadas y comunes para problemas de diseño de software. Estos son el enfoque principal de la guía.
- **Patrones de Arquitectura:** Patrones más universales y de alto nivel, que se pueden implementar en casi cualquier lenguaje y se utilizan para diseñar la arquitectura de una aplicación completa.

Dentro de los **patrones de diseño**, se clasifican por su propósito en tres tipos:

- **Patrones Creacionales:** "Proporcionan mecanismos de creación de objetos que incrementan la flexibilidad y la reutilización de código existente."

- **Patrones Estructurales:** "Explican cómo ensamblar objetos y clases en estructuras más grandes a la vez que se mantiene la flexibilidad y eficiencia de la estructura."
- **Patrones de Comportamiento:** "Se encargan de una comunicación efectiva y la asignación de responsabilidades entre objetos."

3. Patrones de Diseño Creacionales en Java

La guía profundiza en cuatro patrones de diseño creacionales comúnmente utilizados en Java:

3.1. Factory Method (Método Fábrica o Constructor Virtual)

- **Propósito:** "Se utiliza para crear objetos sin especificar la clase exacta del objeto que se creará." Proporciona una interfaz para crear objetos, delegando la implementación específica a las subclases.
- **Beneficio:** Permite agregar nuevas clases de objetos sin modificar el código existente, aumentando la flexibilidad y extensibilidad al "desactivar el código que crea los objetos del código que los utilizan."
- **Implementación clave:** Una clase abstracta 'Creator' define el método 'createProduct()', y las subclases concretas lo implementan para devolver instancias de productos concretos. La interfaz 'Product' define los métodos que deben implementar los productos concretos.

3.2. Abstract Factory (Fábrica Abstracta)

- **Propósito:** "Busca agrupar un conjunto de clases que tiene un funcionamiento en común llamadas familias, las cuales son creadas mediante un Factory". "Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas."
- **Utilidad:** Especialmente útil cuando se requieren "ciertas familias de clases para resolver un problema, sin embargo, puede que se requieran crear implementaciones paralelas de estas clases para resolver el mismo problema, pero con una implementación distinta." Facilita la creación de sistemas independientes de cómo se crean, componen y representan sus productos.

3.3. Constructor (Builder)

- **Propósito:** "Se utiliza para construir objetos complejos paso a paso."
- **Beneficio:** "Separa la construcción de un objeto complejo de su representación, lo que permite que el mismo proceso de construcción cree diferentes representaciones del objeto." Proporciona una forma flexible de crear distintas representaciones de un objeto.

3.4. Singleton

- **Propósito:** "Este patrón garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella."

- **Implementación clave:** Se recomienda "ocultar el constructor, declarar el método 'getInstance' como estático y usar mecanismos que aseguran la atomicidad del método 'getInstance' en entornos concurrentes."
- **Aplicación en la Actividad Formativa:** La actividad formativa de la Semana 1 requiere implementar el patrón Singleton para un "sistema de descuentos que aplique diferentes promociones a los productos". Se debe crear una instancia única de la clase DiscountManager que será responsable de aplicar los descuentos. Para ello, se deberá:
- Crear una clase llamada "Singleton" (o DiscountManager según el contexto del caso) con un constructor privado.
- Declarar una instancia privada, estática y final de la propia clase.
- Crear un método estático getInstance() para obtener la instancia única de la clase.
- Asegurarse de que la clase Singleton solo tenga una instancia y sea accesible desde cualquier parte de la aplicación.

4. Herramientas y Recursos

Para la implementación y práctica de estos patrones, se recomienda:

- **Java Development Kit (JDK):** Para el entorno de desarrollo Java. Se sugiere descargar el x64 Installer.
- **Apache NetBeans:** Un entorno de desarrollo integrado (IDE). Se recomienda la versión 18 por ser la más estable.

La documentación también proporciona diversos **Links de Interés** y **Videos** para profundizar en los conceptos y la instalación de las herramientas.

5. Resultado de Aprendizaje y Actividad

El objetivo de aprendizaje principal para el estudiante es:

- **RA1:** Analizar la importancia de la arquitectura de software en distintos niveles de sistemas informáticos, incluyendo impactos en el desarrollo de componentes, con énfasis en buenas prácticas de la industria.
- **IL1 (Indicador de Logro):** Utilizar diversos conceptos y formas de patrones de diseño como base para la creación de un proyecto de software en Java.

La actividad formativa de la semana 1, "Aplicando conceptos básicos de diseños y formatos simples", se realizará en parejas y consistirá en iniciar un proyecto de Java, seleccionando e integrando el patrón de diseño Singleton como solución a un caso específico de un sistema de pedidos online y gestión de descuentos. La entrega final será un archivo .jar. La evaluación se basará en criterios como la correcta creación de la clase DiscountManager con constructor privado, la declaración de una instancia privada, estática y final, y la creación de un método estático getInstance().